- Dokumentacja Projektu: Inteligentny Asystent Szachowy
  - Przewodnik dla Nowych Programistów (Onboarding)
  - 1. Zespół AVS (Wizja Komputerowa i AI)
    - Jak to działa pod maską:
    - Gdzie szukać kodu (Kluczowe pliki):
    - Stan obecny i wyzwania:
  - 2. Zespół GLAS (Logika Gry i API)
    - Jak to działa pod maską:
    - Gdzie szukać kodu (Kluczowe pliki):
    - Stan obecny i wyzwania:
  - 3. Zespół Frontend (Interfejs Użytkownika)
    - Jak to działa pod maską:
    - Gdzie szukać kodu (Kluczowe pliki):
    - Stan obecny i wyzwania:
  - 4. Zespół DevOps (Infrastruktura i Docker)
    - Jak to działa pod maską:
    - Gdzie szukać kodu (Kluczowe pliki):
    - Stan obecny i wyzwania:

Dokumentacja Projektu: Inteligentny Asystent Szachowy

Przewodnik dla Nowych Programistów (Onboarding)

Witaj w zespole! Ten dokument został stworzony, aby pomóc Ci zrozumieć architekturę i kod naszego systemu, nawet jeśli widzisz go po raz pierwszy.

System składa się z trzech głównych mikroserwisów, które komunikują się ze sobą, aby przenieść fizyczną partię szachów na wirtualną planszę w przeglądarce. Poniżej znajdziesz szczegółowy podział projektu z perspektywy czterech kluczowych ról. Zobaczysz, co zostało zrobione, jak to działa i gdzie w kodzie szukać poszczególnych mechanizmów.

---

1. Zespół AVS (Wizja Komputerowa i AI)

Katalog roboczy: `chess-vision-service/` **Odpowiedzialność:** Pobieranie obrazu z kamer, rozpoznawanie figur za pomocą AI i wysyłanie danych dalej.

Jak to działa pod maską:

Kiedy uruchamiasz system, plik `main.py` inicjuje pętlę (Vision Loop). Pobiera on klatki z kamer (lub z folderu `mock-data/` w trybie testowym). Następnie każda klatka jest "prostowana" za pomocą transformacji perspektywy (PerspectiveManager), aby szachownica zawsze wyglądała jak idealny kwadrat z lotu ptaka. Na tak przygotowany obraz wkracza YOLOv8 (`PieceDetector`), który szuka figur. Na koniec wyniki z kilku kamer są łączone w jeden spójny stan planszy (`FusionManager`).

Gdzie szukać kodu (Kluczowe pliki):

- **`main.py`**: Serce serwisu. Tutaj znajduje się pętla główna (`vision_loop()`), serwer MJPEG przesyłający obraz na frontend (`/stream`) oraz zarządzanie kamerami (`MultiCaptureService`).
- **`detector.py`**: Kod odpowiedzialny za model AI. Jeśli chcesz zobaczyć, jak mapujemy współrzędne pikseli na pola szachowe (np. 'e4'), szukaj funkcji `identify_square`.
- **`fusion.py`**: Algorytm łączący dane z wielu kamer, rozwiązujący problem zasłaniania się figur.
- **`calibrate.py`**: Narzędzie do ręcznego oznaczania rogów szachownicy na obrazie z kamery.

Stan obecny i wyzwania:

- **Zrobione**: Skalowanie obrazów, auto-kalibracja (wyszukiwanie rogów co 30 sekund), MJPEG stream z nałożonymi etykietami figur, tryb symulacji z plików wideo/zdjęć.
- **Problemy**: Wymagane jest poprawienie detekcji rogów przy bardzo słabym oświetleniu.

- **Co dalej**: Wymiana standardowego modelu YOLO na model wytrenowany specjalnie na naszych, fizycznych figurach szachowych.

---

2. Zespół GLAS (Logika Gry i API)

Katalog roboczy: `chess-logic-api/` **Odpowiedzialność:** Utrzymanie wirtualnego stanu gry, sprawdzanie poprawności ruchów i powiadamianie interfejsu użytkownika.

Jak to działa pod maską:

GLAS nasłuchuje na kolejce wiadomości (RabbitMQ). Kiedy AVS wykryje figury na fizycznej planszy, wysyła je tutaj. Zamiast ślepo wierzyć kamerze (która może się mylić, gdy np. ręka gracza zasłoni planszę), GLAS posiada "Silnik Różnicowy" (Difference Engine). Czeka on, aż obraz ustabilizuje się na co najmniej 2 klatki (filtracja szumów). Następnie porównuje to, co "widzi" kamera, z tym, co jest na wirtualnej planszy, i próbuje zgadnąć, jaki ruch wykonał gracz (np. pionek zniknął z e2 i pojawił się na e4).

Gdzie szukać kodu (Kluczowe pliki):

- `main.py`: Ten plik to potężny kombajn. Zawiera:
  - `GameSession`: Klasa zarządzająca stanem (przechowuje wirtualną deskę).
  - `process_vision_update`: Serce logiki wnioskowania ruchów i odrzucania błędnych odczytów (szumów).
  - `websocket_endpoint`: Miejsce, do którego podłącza się przeglądarka internetowa, aby na żywo odbierać FEN (zapis stanu szachownicy).
  - `start_rabbitmq_consumer`: Wątek odbierający paczki danych od serwisu wizyjnego (AVS).

Stan obecny i wyzwania:

- **Zrobione**: Niezawodne WebSockety (odporne na zrywanie połączeń), detekcja prostych ruchów (Piece Inference), Tryb Setup (obsługa niepełnej planszy bez crashowania systemu).

- **Problemy**: Złożone ruchy (np. roszada czy bicie w przelocie) nie są jeszcze rozpoznawane przez automatyczny inferencer.
- **Co dalej**: Integracja z silnikiem szachowym (Stockfish), aby grać przeciwko sztucznej inteligencji, oraz rozwój rozpoznawania roszad.

---

3. Zespół Frontend (Interfejs Użytkownika)

Katalog roboczy: `chess-web-ui/` **Odpowiedzialność:** Przejrzyste prezentowanie danych graczowi w przeglądarce.

Jak to działa pod maską:

Frontend to aplikacja React (zbudowana w Vite). Działa całkowicie pasywnie – łączy się z backendem (GLAS) przez WebSocket i nasłuchuje komunikatów. Kiedy przyjdzie nowy stan gry (zapisany jako ciąg znaków FEN), aplikacja natychmiast aktualizuje wirtualną szachownicę. Równolegle aplikacja pobiera strumień wideo na żywo bezpośrednio z serwisu wizyjnego (AVS), co pozwala na podgląd "oczu" systemu.

Gdzie szukać kodu (Kluczowe pliki):

- `src/App.tsx`: Główny widok układający elementy na ekranie (deska po lewej, wideo po prawej). Znajdziesz tu też logikę WebSocketów i auto-reconnect (automatyczne wznawianie połączenia).
- `src/components/ChessBoardView.tsx`: Komponent wirtualnej szachownicy. Zwróć uwagę na to, że przy wymuszonym odświeżaniu ignoruje on wewnętrzne błędy biblioteki `chess.js`, aby wyświetlić planszę nawet wtedy, gdy na obrazie brakuje królów.
- `src/components/VisionPreview.tsx`: Komponent pobierający obraz z kamery (tag `<img>` podpięty pod URL streama).

Stan obecny i wyzwania:

- **Zrobione**: Minimalistyczny i responsywny układ, bezpieczne renderowanie zepsutych stanów planszy (alert "Missing King"), strumieniowanie wideo bez opóźnień.
- **Problemy**: Biblioteka `react-chessboard` może zachowywać się nieprzewidywalnie w kontenerach z restrykcyjnym CSS. Wymagało to ostrożnego podejścia do styli Tailwind (np. usunięcia `aspect-square`).
- **Co dalej**: Dodanie logów ruchów (lista PGN), wskaźniki dla ruchów sztucznej inteligencji.

---

4. Zespół DevOps (Infrastruktura i Docker)

Katalog roboczy: Pliki w głównym katalogu projektu. **Odpowiedzialność:** Łatwe uruchamianie całego systemu jednym poleceniem na dowolnym komputerze.

Jak to działa pod maską:

Każdy z mikroserwisów jest odizolowany w osobnym kontenerze Docker. Rozmawiają one ze sobą poprzez specjalną wirtualną sieć (`chess-net`). Centralnym punktem komunikacyjnym jest RabbitMQ (pośrednik wiadomości). Osobny plik konfiguracyjny do programowania (`docker-compose.dev.yml`) pozwala na edycję plików na Twoim komputerze i natychmiastowe widzenie zmian w działającym systemie bez potrzeby restartu (Hot-Reload).

Gdzie szukać kodu (Kluczowe pliki):

- **`docker-compose.yml`**: Konfiguracja produkcyjna.
- **`docker-compose.dev.yml`**: Konfiguracja dla programistów. Zawiera "wolumeny" (volumes) mapujące Twój lokalny kod do wnętrza kontenerów. Zawiera także reguły `depends_on`, które sprawdzają, czy RabbitMQ "żyje" (Healthcheck), zanim uruchomią API i Wizję.
- **`chess-vision-service/Dockerfile`**: Plik opisujący budowę środowiska Python. Posiada kluczową linijkę z `# syntax=docker/dockerfile:1` oraz `--`

`mount=type=cache`, co sprawia, że pobieranie ciężkich bibliotek AI (YOLO) jest znacznie szybsze na lokalnym komputerze (korzysta z cache).

Stan obecny i wyzwania:

- **Zrobione**: Izolacja usług, potężne przyspieszenie budowania obrazów w Dockerze, hot-reloading dla API i Frontendu.
- **Problemy**: Środowiska "slim" w Pythonie często nie mają bibliotek systemowych do renderowania wideo, co wymagało ręcznego dodawania pakietów (np. `libgl1`, `libgomp1`).
- **Co dalej**: Stworzenie lekkich obrazów produkcyjnych (multistage builds) oraz pełne przepięcie infrastruktury CI/CD w GitHub Actions.